Lecture 11

Machine Learning 2

Intelligent Systems
Vrije Universiteit

Overview

1 Recap

2 Machine Learning Methodology

3 Text Features

3.1 Improving Text Features

4 Machine Learning Evaluation

Part 1: Recap

Machine Learning Recap

1. Supervised learning: labelled input data and specified features
   a. Classification: predict class membership (Boolean/discrete)
   b. Regression: predict numerical value
   c. Ranking: order data
   d. Collaborative filtering: make suggestions based on similar users

      (can be unsupervised)

   Algorithms: Naive Bayes classification, k-Nearest Neighbour

2. Unsupervised learning: unlabelled input data and no specified features
   a. Clustering: group similar objects

Part 2: Machine Learning Methodology

Intelligent Systems
Vrije Universiteit

Supervised Machine Learning Methodology

Data: labeled instances, *e.g.* emails marked spam or not spam

- Training set: what we show the model
- Validation set: held out set
- Test set: what we evaluate the model on

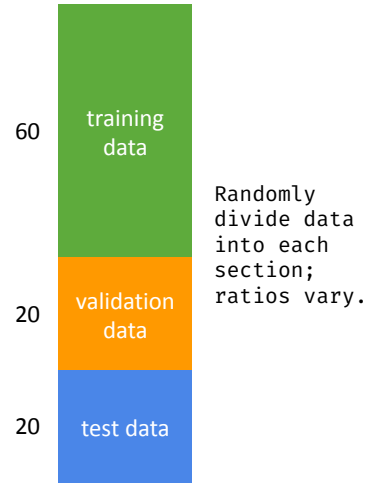
Features: attribute-value pairs which are characteristics of each instance

Experimentation cycle:

- Select hypothesis (tune *hyperparameters* on validation set)
- Compute accuracy on test set
- Important note: never 'peek' at test set

Evaluation: how good is the performance of the model

- Accuracy: fraction of instances predicted correctly



The basis for a machine learning problem is data (e.g. a large collection of emails) for which we have labels (e.g. spam/not spam). The bulk of this data becomes what we call the training set, which is the data we use to train the model on. It is from this data that the model learns the patterns that set apart spam emails from not spam emails in our example. The remainder of the data becomes test data, which is the data on which the model is later tested. It is important that this data is unseen to the model in order to evaluate its performance. In some, but not all, cases, a validation set is also separated out. This dataset is used to tune hyperparameters and to do other things which we expect will improve our model's performance.

Secondly, we need to choose features of the data which we care about. In our email example this could be 'email length', 'frequency of capital letters', 'frequency of typos', etc. Next we train perform experimentation cycles in which we train our model, and finally evaluate the performance of the model on the test set.

Basic Machine Learning Recipe

1. Choose features, or the characteristics of each data point we will use to make a prediction

2. Choose the model class

3. Choose a search method

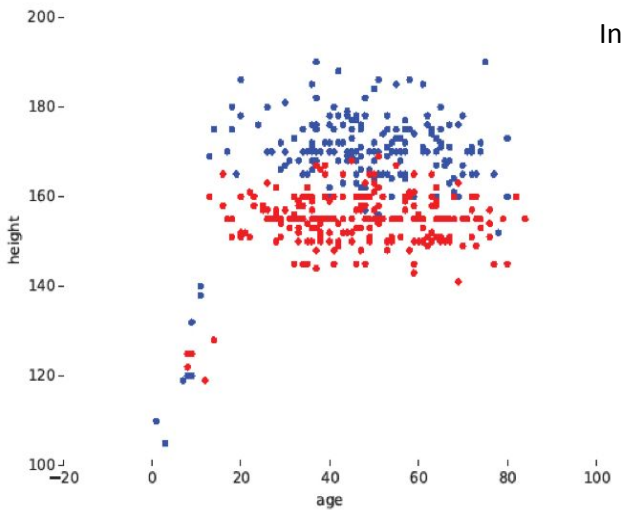
Features

feature vector $\begin{pmatrix} 3 \\ 105 \end{pmatrix}$

tree age	tree height	label (class)
3	105	A
1	110	A
8	120	A
9	120	A
11	124	B
7	127	B
7	130	B
8	130	B
9	132	A
13	133	B
...	...	...

For the purposes of this course we will assume that all features are numeric. There are also algorithms that handle categoric features,but we'll ignore them. A single instance described only by numerical features corresponds to a mathematical object called a **vector** (which is just a list of numbers of a fixed size). If we only have 2 features, we can plot each instance in our dataset in two-dimensional axes. We usually have much more features than that, but to illustrate what classifiers look like, we will use a dataset with two features: our instances are trees and we will measure the age and the height of each.

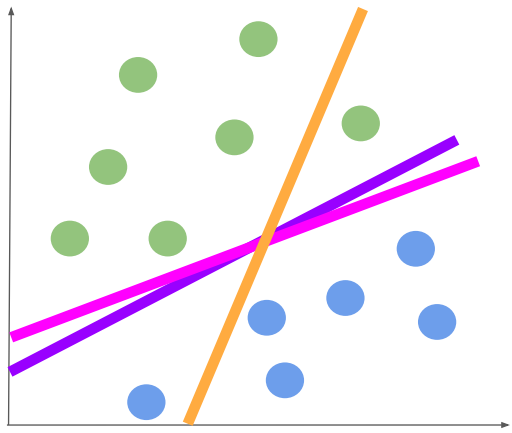
1. Feature Space



In the feature space, every point represents an instance.

Now we have the two features height and age plotted on the x and y axes. Every point in the **feature space** represents an instance, in this case a person.

3. Finding the Best Model



Which classifier is the best divider of the feature space?

Now the question is, how can we divide the feature space in a way that's useful for categorising the instances into correct classes? More on this next lecture.

Part 3: Text Features

Intelligent Systems
Vrije Universiteit

Binary Classification with Text: Spam Emails

Input: email

Output: spam or inbox

Setup:

- Collect large dataset of emails, each labeled
- Note: who will label this data?
- Goal: predict labels of new, future emails

Features: attributes used to make prediction

e.g. words ("FREE!"), patterns ("$dd", CAPS), non-text (SenderInContacts)

✗ Dear Sir.

First, I must solicit your confidence in this transaction, this is by virture of its nature as being utterly confidencial and top secret. …

✗ TO BE REMOVED FROM FUTURE MAILINGS, SIMPLY REPLY TO THIS MESSAGE AND PUT "REMOVE" IN THE SUBJECT.

99 MILLION EMAIL ADDRESSES FOR ONLY $99

✓ Ok, Iknow this is blatantly OT but I'm beginning to go insane. Had an old Dell Dimension XPS sitting in the corner and decided to put it to use, I know it was working pre being stuck in the corner, but when I plugged it in, hit the power nothing happened.

Let's specifically focus on text features using an example of a binary classification task. In this task, the classifier is a detector for the positive class, or when an email is spam. In order to make this prediction, the model learns from many examples of emails with the label 'spam' or 'inbox', and learns which features these examples have. These features may be specific words (e.g. FREE), patterns in the text (e.g. capital letters), or non-text features (e.g. who is the sender?).

Similarity of Text

We have control over which features of a text we're interested in, *e.g.*

- Average word length
- Similar sentiment
- Average sentence length
- Use of punctuation
- Language
- **Occurrence of keywords**
- etc.

In supervised learning, the engineer chooses which features to use in order to measure similarity between two texts. In this course we mostly focus on the occurrence of keywords, or when a word occurs in two texts.

Using keywords as features

1. Convert the text to a set of words
2. Use 1 dimension per unique word
3. Set the dimension to 1 if the word occurs, 0 otherwise

e.g.

	The student learns about machine learning.
1	{The, student, learns, about, machine, learning, .}
2	['and', 'the', 'teacher', 'about', 'student', 'computer', 'learning', 'machine', 'computer', 'learns', …]
3	[0, 1, 0, 1, 1, 0, 1, 1, 0, 1, …]

To extract words from a text, translate the text into a set of all of its words comma-separated. The goal is have a vector of all possible words that could occur in the sentence, and this requires the words to be ordered in a list. The underlined words represent those that occur in the sentence we are looking at. This list is then translated into a list of identical length where every words is a 0 if it does not occur in the sentence and 1 if it does occur.

Feature-Based Similarity

Two texts are similar when their feature vectors are similar. Methods for measuring similarity:

1. Hamming distance

$$\frac{\texttt{\# bits different}}{\texttt{\# bits}}$$

2. Euclidean distance

$$\sqrt{\sum_{i=1}^{n}(q_i - p_i)^2}$$

3. Cosine similarity

$$\frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

Two texts are similar when the constructed feature vectors (the 1s and 0s) are similar. We'll cover three methods of measuring the distance between two vectors.

Hamming Distance

$$\frac{\text{\# bits different}}{\text{\# bits}}$$

Hamming distances represent the number of features you would have to change to convert one data point to another.

e.g. $\text{dist}_{\text{Hamming}}([0,1,1],[1,1,0]) = 2$

$\text{dist}_{\text{Hamming}}([1,1,1],[1,1,0]) = 1$

Hamming distance is the simplest measure. It works by dividing the number of bits (being 0 or 1) that differ by the number of total bits (the length of the vector).

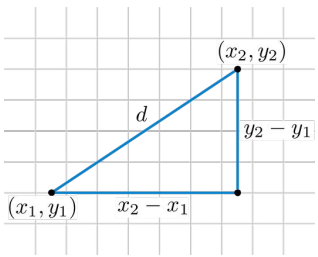
Euclidean Distance

$$\sqrt{\sum_{i=1}^{n}(q_i - p_i)^2}$$

Euclidean distance is the length of the line segment that connects two coordinates in space.

e.g. $\text{dist}_{\text{Euclidean}}([0,1,1], [1,1,0]) = \sqrt{(0-1)^2+(1-1)^2+(0-1)^2}$

$= \sqrt{(1+0+1)} = \sqrt{2}$



Euclidean distance works by summing over the distances between the vector values in the same position squared, all under a square root. Taking the square is useful because it prevents the result from being negative, and we can take the root at the end.

Cosine Similarity

vector • product

$$\frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|} = \frac{\sum_{i=1}^{n} a_i b_i}{\sqrt{\sum_{i=1}^{n} a_i^2} \sqrt{\sum_{i=1}^{n} b_i^2}}$$

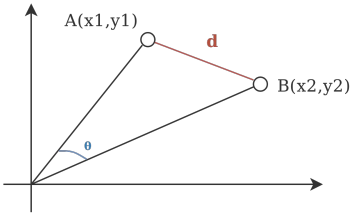
||norm||

Vector notation!

Cosine similarity measures the similarity between two vectors of an inner product space.

e.g.

$$\text{sim}_{\text{Cosine}}([0,1,1],[1,1,0]) = \frac{[0,1,1] \cdot [1,1,0]}{\left\|[0,1,1]\right\| \left\|[1,1,0]\right\|} = \frac{1}{\sqrt{2}\sqrt{2}} = \frac{1}{2}$$



It is equal to the cosine of the angle between two non-zero vectors.

Cosine similarity is the more complex method using the dot product of each vector divided by the product of the lengths of the vectors.

Part 3.1: Improving Text Features

Removing Noise

Noise is characteristics of the data that make it difficult to handle, *e.g.* different capitalisation makes it so that ‘my’ ≠ ‘My’.

- Put all text in lowercase ‘My’ → ‘my’
- Remove punctuation ‘.’ → ‘’
- Stemming: replace words by their stems ‘learning’/’learns’ → ‘learn’

More ways to improve performance:

- Named Entity Recognition (NER): replace words by some identifiers of Named Entities (cities, people, etc.) ‘machine’, ‘learning’ → ‘machine-learning’

If our goal is to create word vectors in order to compare two texts, it is important to recognize two words as the same even if they differ slightly in the way they are written. The first step in cleaning our text data is to put all text in lowercase; this isn’t necessary in every type of machine learning with text but is necessary if we care about the presence of keywords. We can also remove punctuation and perform stemming, which means reducing every word down to its stem regardless of how its conjugated.

Named Entity Recognition is a more complex tool that requires identifying nouns with specific names, such as cities and people. This method is beyond what we will do in this course but is commonly used in practice.

Importance of Words

Words vary in their importance; some words are too rare while others are too common to help the model identify to what class it belongs to.

e.g. “and” tell us nothing about the topic, low importance

1. Removing stop words (and, it, my, the, etc.) declutters word vectors from words with low informational value.
2. Tf-idf is a common tool to calculate word importance (tf * idf)

- Term frequency: how often a word occurs in a document
- Inverse document frequency: how common a word is in the corpus (all documents)

$$\mathrm{idf}(w, C) = \log\left(\frac{|C|}{|\{d \in C : w \in d\}|}\right)$$

Some words have very high information value which makes them good features to use and keep in our vectors. Other words, which we refer to as stop words, can be removed without losing informational value while gaining some efficiency. Common stop words include it, my, and, the, etc. The tf-idf method takes this idea one step further; rather than remove a predefined set of words, it calculates which words are useful and which words are not. Tf-idf is commonly used in practice but we likely won’t be calculating this by hand.

Word Frequency

Some word vectors also track word frequency, in which case a 1 or 0 is replaced with the number of occurrences of a word in a text.

For Named Entity Recognition, we replace the 1s and 0s with the number of occurrences of entities, words, or stems.

	My student likes my teaching.
1	{My, student, likes, my, teaching, .}
2	[‘My’, ‘machine’, ‘student’, ‘likes’, ‘computer’, ‘my’, ‘teaching’, …]
3	[2, 0, 1, 1, 0, 1, 1, …]

Sometimes the engineer may choose to track the word frequency, but we will not be calculating with word frequency in the exam.

Manual Feature Extraction

These methods are part of manual feature extraction, or choosing the features by hand.

- Other machine learning tools can decide for themselves which features are important
- It is usually a good idea to include all features you can think of and let the model decide.

Some other techniques (e.g. deep learning) find features from the patterns in data directly without them ever being explicit.

Choosing which features to measure the similar of two texts on is called manual feature extraction, which is only used in practice for machine learning applications for very specific tasks. Larger models usually decide for themselves which features are important by looking at underlying data structures of each instance, like with deep learning.

Recap

Text → Feature Vector → Machine Learning

- We use domain-specific tricks to get from text to features
- So far we've only seen Naive Bayes and *k*-NN as classifiers
- Note: word count is a discrete feature that we can use directly in the Naive Bayes formula:

$$\text{classify}\,(f_1, \ldots, f_n) = \text{argmax}_c\, p(C = c) \prod_{i=1}^{n} \boxed{p\,(F_i = f_i \mid C = c)}$$

Part 4: Machine Learning Evaluation

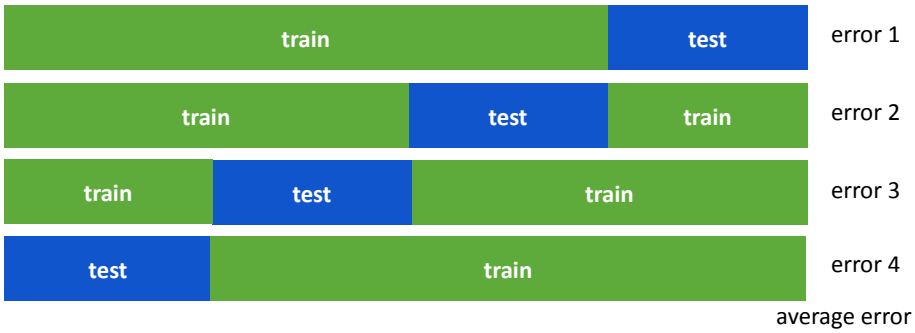
Intelligent Systems
Vrije Universiteit

Once we have trained a model on the training data, we evaluate the model using the test data to see whether it performs well.

Evaluation

Evaluation is the final step in the machine learning process, involving assessing a model's performance by allowing it to make predictions on the test set.

Cross-validation is one method which is useful when dealing with limited data: iteratively split the data into test/train.



Model evaluation is the process of analysing its performance on correctly predicting over the testing data.
An important method for evaluating a model is called **cross-validation**, which is a way of splitting the data into test and train sets iteratively in order to see what the error is for each iteration. Since the data is split differently in each iteration, it is a useful way to see how well a model will generalise to a new dataset.

Evaluation

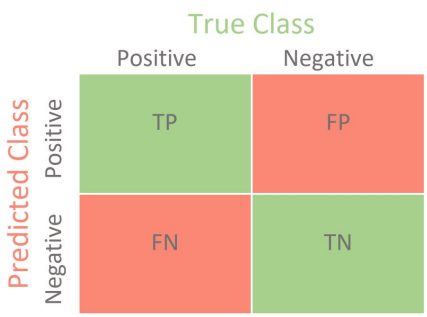
Sometimes, the **training** and **test** split is standardized (especially in research).
For research competitions, the test set is even secret!

In those cases, there is often also a **validation** set to use as your "own" test set. This helps you to find the best model performance on data that you haven't trained on.

Model evaluation is the process of analysing its performance on correctly predicting over the testing data.
An important method for evaluating a model is called **cross-validation**, which is a way of splitting the data into test and train sets iteratively in order to see what the error is for each iteration. Since the data is split differently in each iteration, it is a useful way to see how well a model will generalise to a new dataset.

Evaluating Classification

Confusion matrices are useful summaries of performance on the test data.



accuracy = (TP + TN) / N

error = 1 - accuracy = (FP + FN) / N

recall = TP / (TP + FN)

a.k.a. True Positive Rate, sensitivity

precision = TP / (TP + FP)

a.k.a. positive predictive value (PPV)

What are the metrics we can use to evaluate a model's performance. A **confusion matrix** is used to evaluate performance of a classification algorithm; it shows us true and false positives, and true and false negatives. Let's look at an example of emails labeled by a model:

True positive: spam labeled spam
False positive: *not* spam labeled spam
True negative: *not* spam labeled not spam
False negative: spam labeled not spam

We can use these values to calculate accuracy, error, recall, and precision.

Evaluating Classification *Example*

email	true label	predicted label
1	spam	spam
2	¬spam	spam
3	¬spam	¬spam
4	¬spam	¬spam

	Actual Pos	Actual Neg
Predicted Pos	1	1
Predicted Neg	0	2

accuracy = (TP + TN) / N = ?

error = (FP + FN) / N = ?

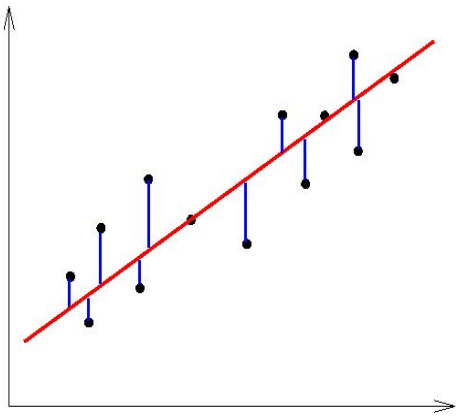
recall = TP / (TP + FN) = ?

precision = TP / (TP + FP) = ?

Try this example.

Evaluating Regression

The mean squared error (MSE) function



$$\mathrm{mse}(p) = \Sigma_i (f_p(x_i) - y_i)^2$$

For each point, take its distance from the prediction, square, and sum together.

For regression we can't use the same method, because a predicted number is not necessarily just correct or incorrect; a prediction can be closer or further from the correct label. For regression we use mean squared error, which takes the distance between the predicted number and the true number.

Evaluating Ranking

Precision@k measure is the fraction of relevant items in the top k recommendations.

$$\text{Precision@}k = \frac{true\ positives\ @k}{(true\ positives\ @k) + (false\ positives\ @k)}$$

To choose *k* (*e.g.* *k*= 1, 2, …) or the top-*k* hit rate:

1. Take set of items a person likes
2. Remove one item
3. Recommend.
4. Is the item in the top-*k* items recommended?
5. Aggregate score

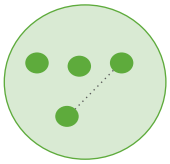
Precision @k measure is an evaluation metric for ranking and recommender systems. The *k* value is a chosen number that represents a certain number of top recommendations. So, true positive @k means that number of correct rankings within the top k items.

Evaluating Clustering

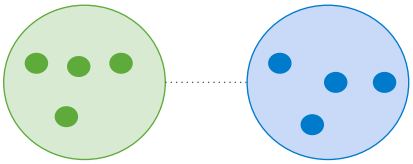
Internal criterion dictates that good clustering will produce high-quality cluster in which:

1. Intra-class (cluster) similarity is high
2. Inter-class similarity is low
3. Measured quality of clustering depends on similarity measure used

External criterion: compare results of cluster analysis to externally known result, such as class labels



INTRA-CLUSTER



INTER-CLUSTER

Since clustering is an unsupervised learning task, it requires that we set an internal criterion in order to evaluate its performance. An internal criterion dictates that good clustering produces clusters with high intra- and low inter-class similarity. Essentially, data points within the same cluster as x must be similar, and all others must be dissimilar. This is not exam material but good to think about.

Intelligent Systems Framework

Efficient, simple, correct, well-understood, generic	Find best decision based on experience.	Weeks 6 & 7
	Find best decision with knowledge & reason.	Weeks 3, 4, 5
	Find best decision in multiple agent setting.	Weeks 1 & 2
	Find best decision with heuristics.	
	Systematically find all possible solutions.	

Progress in the intelligent systems framework.

Summary

Machine learning projects start with data, separated into subsets.

Predictions are made about instances based on their characteristics, or their **features**. Specifically for text, we can create feature vectors with 1s for word occurrences and 0s for non-occurrences.

We can clean text data by lowercasing, removing punctuation and stemming, or choose important words with tf-idf.

There are different **evaluation** methods for different tasks, which allow us to compare performance of models on test data.

Thank you for your attention and have a great day.